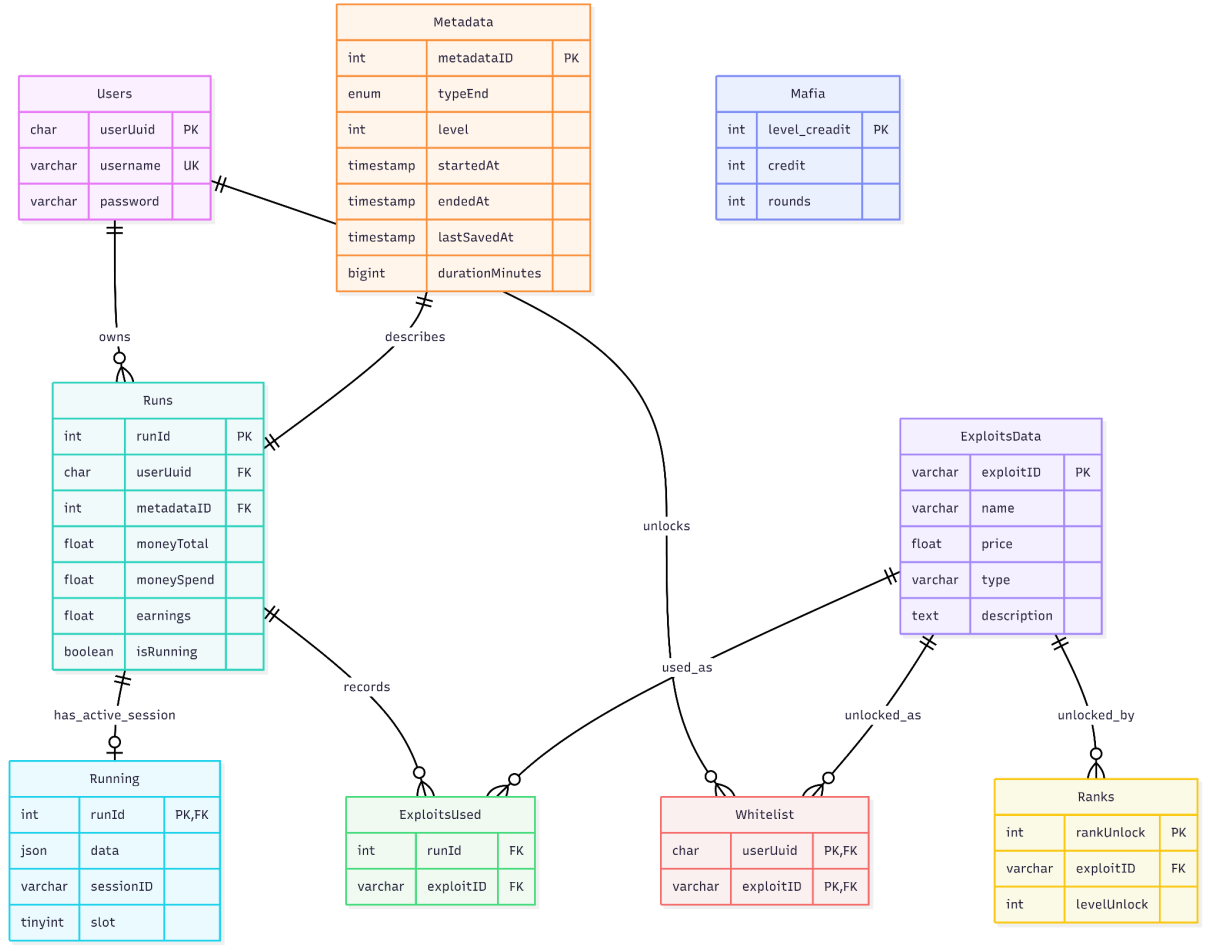
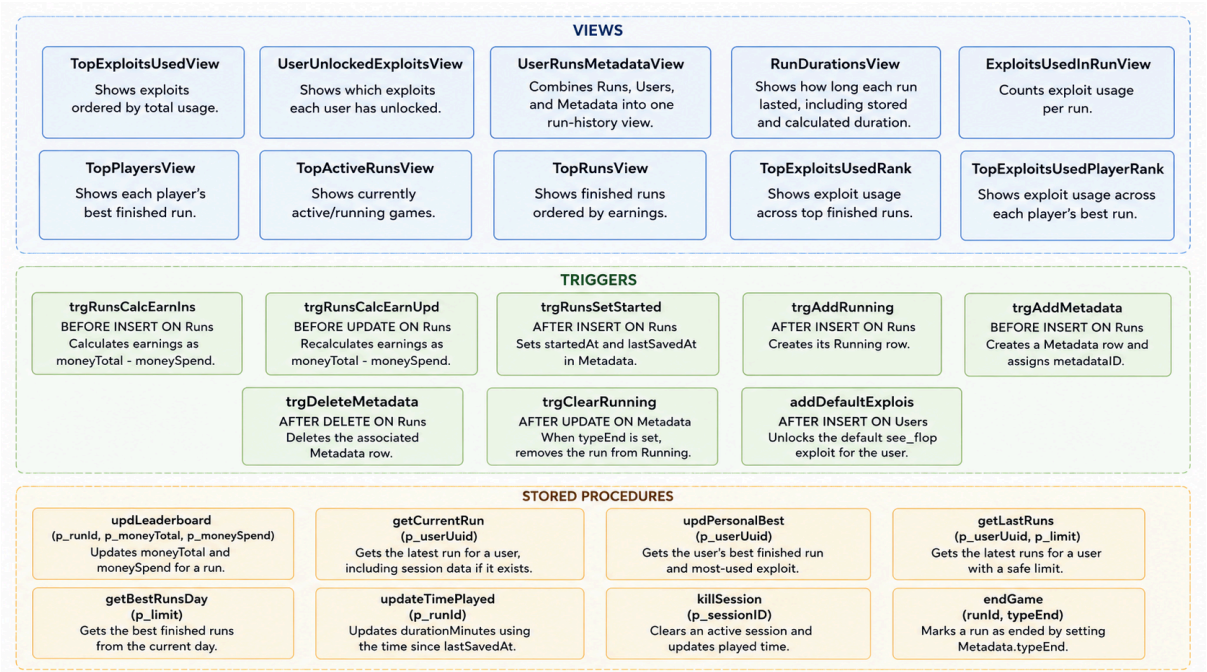
General view

Diagram Schema Related Views, Triggers and Stored Proceduredures





Views

1. TopExploitsUsedView: ranks exploits by total times used.
2. UserUnlockedExploitsView: shows which exploits each user has unlocked.

3. UserRunsMetadataView: joins users, runs, and metadata to show run history.
4. RunDurationsView: shows how long each run lasted, including stored and calculated duration.
5. ExploitsUsedInRunView: counts how many times each exploit was used in each run.
6. TopPlayersView: shows each player's best finished run.
7. TopActiveRunsView: shows active/running games ordered by earnings.
8. TopRunsView: shows finished runs ordered by earnings.
9. TopExploitsUsedRank: ranks exploit usage across top finished runs.
10. TopExploitsUsedPlayerRank: ranks exploit usage across each player's best run.

Triggers

1. trgRunsCalcEarnIns: calculates earnings before inserting a run.
2. trgRunsCalcEarnUpd: recalculates earnings before updating a run.
3. trgRunsSetStarted: sets startedAt and lastSavedAt after a run is created.
4. trgAddRunning: creates a Running row after a run is created.
5. trgAddMetadata: creates metadata before inserting a run.
6. trgDeleteMetadata: deletes metadata when a run is deleted.
7. trgClearRunning: removes active session data when a run ends.
8. addDefaultExplois: gives new users the default see_flop exploit.

Stored Procedures

1. updLeaderboard: updates moneyTotal and moneySpend.
2. getCurrentRun: gets the latest run for a user.
3. updPersonalBest: gets a user's best finished run.
4. getLastRuns: gets recent runs for a user.
5. getBestRunsDay: gets today's best finished runs.
6. updateTimePlayed: updates played time for a run.
7. killSession: clears an active session and saves elapsed time.
8. endGame: marks a run as ended.

Justification

The database design separates the main concepts of the game into independent tables to reduce duplication and keep the data consistent. Users stores account information, Runs stores each gameplay attempt, Metadata stores timing and ending details, and ExploitsData stores the catalog of available exploits. This keeps each table focused on one responsibility.

Primary keys are used to uniquely identify each entity. For example, Users.userUuid uniquely identifies a player, Runs.runId uniquely identifies a game run, Metadata.metadataID uniquely identifies the run metadata, and ExploitsData.exploitID uniquely identifies each exploit. Composite keys are used in relationship tables like Whitelist, where the combination of userUuid and exploitID prevents the same exploit from being unlocked twice by the same user.

Foreign keys are placed where the game logic requires a relationship. A Run belongs to a User, so Runs.userUuid references Users.userUuid. A Run has metadata, so Runs.metadataID references Metadata.metadataID. Active saved sessions belong to a run, so Running.runId references Runs.runId. Exploit usage is modeled through ExploitsUsed,

which connects runs with exploits, and unlocked exploits are modeled through Whitelist, which connects users with exploits.

The views are justified because they simplify repeated queries used by the application and leaderboard. For example, TopRunsView, TopPlayersView, and TopActiveRunsView prepare leaderboard-ready data without requiring the application to repeat complex joins. RunDurationsView centralizes the logic for calculating how long each run lasted, including active games and completed games.

Triggers are used to automate values that should always stay consistent. Earnings are automatically calculated from moneyTotal - moneySpend, metadata is automatically created when a run is inserted, and running session data is automatically removed when a game ends. This reduces the chance of inconsistent data caused by forgetting to update related tables manually.

Stored procedures are used for common database operations such as getting the current run, updating leaderboard values, ending a game, killing a session, and calculating time played. This keeps important database behavior centralized and reusable.

Overall, the schema is normalized because entity data, relationship data, runtime session data, and statistical/leaderboard logic are separated. The model supports the core game flow: users start runs, runs generate metadata and stats, exploits can be unlocked or used, and views/procedures expose clean data for gameplay, summaries, and leaderboards.

The schema satisfies Third Normal Form (3NF). It is in 1NF because all attributes are atomic, in 2NF because all non-key attributes depend on the full primary key, and in 3NF because non-key attributes do not depend on other non-key attributes. Entity data is separated into tables such as Users, Runs, Metadata, and ExploitsData, while many-to-many relationships are handled through Whitelist and ExploitsUsed. This reduces redundancy and prevents insertion, update, and deletion anomalies.